

Python Libraries for DSML CheatSheet

Table of Contents

1. NumPy

- Why Use NumPy?
- Getting Started with NumPy
- NumPy Arrays and Operations
- Data Types in NumPy
- Array Reshaping
- Joining and Splitting Arrays
- Sorting and Searching Arrays
- Mathematical Functions
- Best Practices for NumPy

2. Pandas

- What Can Pandas Do?
- Data Structures in Pandas
- Data Cleaning with Pandas
- Filtering and Modifying DataFrames
- Data Analysis with Pandas
- Combining DataFrames
- Best Practices for Pandas

3. Matplotlib

- Types of Plots in Matplotlib
- Customizing Plots
- Subplots in Matplotlib
- Best Practices for Matplotlib

4. Matplotlib Best Practices

a. How Does Matplotlib Help Me?

5. Python Tools and IDEs

6. Additional Resources

7. Conclusion

NumPy



NumPy (numerical python) is an open source Python library used for working with arrays.

Provides support for arrays, matrices, and a large collection of mathematical functions.

Why Use NumPy?

- In Python we have lists that serve the purpose of arrays, but they are slow to process.
- NumPy aims to provide an array object that is up to 50x faster than traditional Python lists.
- The array object in NumPy is called `ndarray`, it provides a lot of supporting functions that make working with `ndarray` very easy.

Why is NumPy Faster Than Lists?

NumPy arrays are stored at one continuous place in memory unlike lists, so processes can access and manipulate them very efficiently.

This behavior is called locality of reference in computer science.

Getting Started with NumPy

1. Import NumPy

```
In [1]: # Importing NumPy
import numpy as np
```

2. NumPy Creating Arrays

```
In [2]: # Creating a 1D array
arr = np.array([1, 2, 3, 4])
arr
```

```
Out[2]: array([1, 2, 3, 4])
```

3. Finding Dimensions in Arrays

NumPy Arrays provides the **ndim** attribute that returns an integer that tells us how many dimensions the array have.

```
In [3]: # Finding the dimensions of an array
a = np.array([[1,2,3],[4,5,6]],[[1,2,3],[4,5,6]])
a.ndim
```

```
Out[3]: 3
```

4. Higher Dimensional Arrays

When the array is created, you can define the number of dimensions by using the **ndmin** argument.

```
In [4]: arr = np.array([1, 2, 3, 4], ndmin=5)

print(arr)
print('Number of dimensions:', arr.ndim)
```

```
[[[[[1 2 3 4]]]]]
Number of dimensions: 5
```

5. NumPy Array Indexing

Array indexing is the same as accessing an array element.

```
In [5]: # Array indexing  
  
arr = np.array([1, 2, 3, 4])  
arr[0]
```

```
Out[5]: 1
```

6. NumPy Array Slicing

Slicing in python means taking elements from one given index to another given index.

- We pass a slice instead of an index like this: `[start:end]`.
- We can also define the step, like this: `[start:end:step]`.
- If we don't pass, it's considered 0.
- If we don't pass end its considered length of array in that dimension.
- If we don't pass step it's considered 1.

```
In [6]: # Slicing an array

arr = np.array([1, 2, 3, 4, 5, 6, 7])
arr[1:5:2]

Out[6]: array([2, 4])
```

7. NumPy Data Types

Below is a list of all data types in NumPy and the characters used to represent them.

- **i** - integer
- **b** - boolean
- **u** - unsigned integer
- **f** - float
- **c** - complex float
- **m** - timedelta
- **M** - datetime
- **O** - object
- **S** - string

- **U** - unicode string
- **V** - fixed chunk of memory for other type (void)

a. Checking the Data Type of an Array

The NumPy array object has a property called **dtype** that returns the data type of the array:

```
In [7]: # Checking the data type of elements in the array
```

```
arr = np.array([1, 2, 3, 4])  
arr.dtype
```

```
Out[7]: dtype('int32')
```

8. NumPy Array Shape

The shape of an array is the number of elements in each dimension.

a. Get the Shape of an Array

NumPy arrays have an attribute called **shape** that returns a tuple with each index having the number of corresponding elements.


```
In [8]: # Shape of an array

arr = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
arr.shape
```

```
Out[8]: (2, 4)
```

9. NumPy Array Reshaping

a. Reshaping arrays

- Reshaping means changing the shape of an array.

Example: Reshape From 1-D to 3-D

```
In [9]: # Reshaping an array

arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
new_arr = arr.reshape(2,3,2)
new_arr
```

```
Out[9]: array([[ 1,  2],
               [ 3,  4],
               [ 5,  6]],

              [[ 7,  8],
               [ 9, 10],
               [11, 12]])
```

b. Flattening the arrays

- Flattening array means converting a multidimensional array into a 1D array.

We can use `reshape(-1)` to do this.

```
In [10]: # Flattening the array

arr = np.array([[1, 2, 3],[4, 5, 6]])
new_arr = arr.reshape(-1)
new_arr
```

```
Out[10]: array([1, 2, 3, 4, 5, 6])
```

10. Joining NumPy Arrays

Joining means putting contents of two or more arrays in a single array.

We pass a sequence of arrays that we want to join to the `concatenate()` function, along with the axis. If the axis is not explicitly passed, it is taken as 0.

```
In [11]: # Joining two arrays along axis 0

arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
joined_arr = np.concatenate((arr1, arr2))
joined_arr
```

```
Out[11]: array([1, 2, 3, 4, 5, 6])
```

11. Splitting NumPy Arrays

Joining merges multiple arrays into one and Splitting breaks one array into multiple.

We use `array_split()` for splitting arrays, we pass it the array we want to split and the number of splits.

```
In [12]: # Splitting an array

arr = np.array([1, 2, 3, 4, 5, 6])
new_arr = np.array_split(arr, 3)
new_arr

Out[12]: [array([1, 2]), array([3, 4]), array([5, 6])]
```

12. Searching Arrays

- To search an array, use the **where()** method.
- It returns the indexes that get a match.

```
In [13]: # Searching arrays

arr = np.array([1, 2, 3, 4, 5, 4, 4])
x = np.where(arr == 4)
print(x)

(array([3, 5, 6], dtype=int64),)
```

13. Sorting Arrays

The NumPy ndarray object has a function called **sort()**, that will sort a specified array.

```
In [14]: # Sorting an array

arr = np.array([3, 4, 0, 1])
np.sort(arr)
```

```
Out[14]: array([0, 1, 3, 4])
```

Note: This method returns a copy of the array, leaving the original array unchanged.

You can also sort arrays of strings, or any other data type.

14. Broadcasting

Broadcasting: NumPy's ability to perform element-wise operations on arrays of different shapes.

Rules for Broadcasting:

- The smaller array is “broadcast” across the larger array.
- Two arrays are compatible for broadcasting if:
 - They have the same shape.
 - One of the arrays has a dimension of size 1 in a particular axis.

```
[3]: arr_broadcast = arr_1d + 5 # Adds 5 to each element
arr_broadcast
```

```
[3]: array([6, 7, 8, 9])
```

15. Filtering Arrays

- Use boolean indexing to filter elements.

```
[4]: arr_filtered = arr_1d[arr_1d > 2] # Returns elements greater than 2
arr_filtered
```

```
[4]: array([3, 4])
```

- Combined Conditions

```
[5]: arr_combined = arr_1d[(arr_1d > 2) & (arr_1d < 5)] # Elements > 2 and < 5
arr_combined
```

```
[5]: array([3, 4])
```

16. Joining and Splitting Arrays

- **Joining Arrays:** Use `np.concatenate()` to join two or more arrays.

```
[7]: arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
arr_combined = np.concatenate((arr1, arr2)) # Output: [1, 2, 3, 4, 5, 6]
arr_combined
```

```
[7]: array([1, 2, 3, 4, 5, 6])
```

- **Splitting Arrays:** Use `np.array_split()` to split an array into multiple sub-arrays.

```
[9]: arr_split = np.array_split(arr_1d, 3) # Splits into 3 sub-arrays
arr_split
```

```
[9]: [array([1, 2]), array([3]), array([4])]
```

17. Searching Arrays

- `np.where()`: Returns the indices of elements that match the condition.

```
[10]: indices = np.where(arr_1d > 2) # Finds elements greater than 2
      indices
```

```
[10]: (array([2, 3], dtype=int32),)
```

18. Sorting Arrays

- Use `np.sort()` to sort elements.

```
[11]: sorted_arr = np.sort(arr_1d) # Sorts the array
      sorted_arr
```

```
[11]: array([1, 2, 3, 4])
```

19. Mathematical Functions

- Sum:

```
[12]: arr_sum = np.sum(arr_1d) # Returns the sum of all elements
      arr_sum
```

```
[12]: 10
```

- **Mean:**

```
[13]: arr_mean = np.mean(arr_1d) # Returns the mean of the elements
      arr_mean
```

```
[13]: 2.5
```

- **Standard Deviation:**

```
[14]: arr_std = np.std(arr_1d) # Returns the standard deviation
      arr_std
```

```
[14]: 1.118033988749895
```

20. Advanced Topics

- **Vectorized Operations:** Perform operations directly on arrays without explicit loops, enhancing performance.
- **Array Broadcasting:** Allows efficient element-wise operations on arrays with different shapes.

```
[16]: arr_2d = np.array([[1, 2, 3], [4, 5, 6]])
      arr_broadcast = arr_2d + 5
      arr_broadcast
```

```
[16]: array([[ 6,  7,  8],
            [ 9, 10, 11]])
```

NumPy Best Practices

- Use **broadcasting** to avoid explicit loops, making code more efficient.

- **Filter arrays** using boolean indexing to extract elements based on conditions.
- Always be mindful of **array shapes** when performing operations to avoid errors.

Pandas



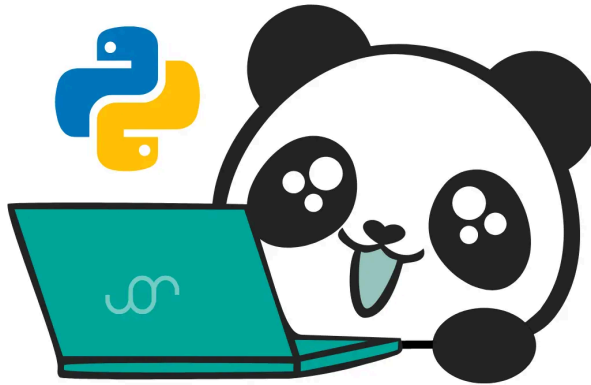
Pandas is a powerful Python library designed for data manipulation and analysis. It provides tools to efficiently work with structured data, such as:

- **Analyzing:** Identifying correlations, averages, min/max values.
- **Cleaning:** Handling missing or irrelevant data.
- **Exploring:** Visualizing and summarizing data.
- **Manipulating:** Modifying, merging, or filtering datasets.

What Can Pandas Do?

Pandas gives you answers about the data. Like:

- Is there a correlation between two or more columns?
- What is the average value?
- Max value?
- Min value?



Pandas are also able to delete rows that are not relevant, or contain wrong values, like empty or NULL values. This is called *cleaning* the data.

Getting Started with Pandas

Import Pandas

```
In [15]: # Importing Pandas  
import pandas as pd
```

Data Structures in Pandas

1. Pandas Series

- A **Series** is similar to a column in a table and is a one-dimensional array capable of holding any data type.

Index	Data
0	Apple
1	Orange
2	Banana
3	Grapes

Example:

```
In [16]: # Creating a Pandas DataFrame

calories = {"day1": 420, "day2":380, "day3":390}
myvar = pd.Series(calories, index = ["day1", "day2"])
myvar

Out[16]: day1    420
         day2    380
         dtype: int64
```

2. DataFrames

A **DataFrame** is a two-dimensional table of data, with rows and columns. It is essentially a collection of Series objects, where each column represents a Series.

Series A		Series B		DataFrame		
	apples		oranges		apples	oranges
0	3	+	0	=	0	0
1	2		3		1	3
2	0		7		2	7
3	1		2		3	2

Example:

- Create a DataFrame from two Series:

```
In [17]: data = {
    "calories" : [420, 380, 390],
    "duration" : [50, 40, 45]
}

myvar = pd.DataFrame(data)
myvar
```

Out[17]:

	calories	duration
0	420	50
1	380	40
2	390	45

Getting Started with DataFrame

1. Filtering DataFrame

Using loc methods

Pandas use the **loc** attribute to return one or more specified row(s)

	column index		0	1	2
row index	label		Name	Age	Education
0	March	2			
1	April	3			
2	May	4			
3	June	5			
4	July	6			

Example:

Return row 0 and 1:

```
In [18]: #use a list of indexes:  
print(df.loc[[0, 1]])
```

2. Using iloc methods

The **iloc** function in Python is part of the pandas library and is used for integer-location-based indexing and selection by

position. It allows you to select rows and columns in a DataFrame by their integer positions.

The `iloc` function is particularly useful when you need to access data by its position in the DataFrame rather than by labels.

Basic Syntax:

```
In [ ]: dataframe.iloc[row_index, column_index]
```

- `row_index`: Integer, list of integers, slice object, or boolean array that specifies which rows to select.
- `column_index`: Integer, list of integers, slice object, or boolean array that specifies which columns to select.
- Note: When slicing, the *to* index is *excluded* from the result.

```
In [19]: # Sample dataframe

df = pd.DataFrame({
    'A': [1, 2, 3, 4],
    'B': [5, 6, 7, 8],
    'C': [9, 10, 11, 12]
})
```

```
In [20]: # Selecting the first two rows and the first two columns

df.iloc[0:2, 0:2]
```

Out[20]:

	A	B
0	1	5
1	2	6

3. Read CSV Files

```
In [ ]: # Reading CSV files

df = pd.read_csv('data.csv')
print(df.to_string())
```

4. Viewing the Data

One of the most used methods for getting a quick overview of the DataFrame, is the `head()` method.

- a. `head()`: Provides a quick overview of the top rows in the DataFrame.

```
In [ ]: # Viewing the data
df.head()
```

- b. `info()`: Displays a concise summary of the DataFrame.

```
[ ]: df.info()
```

5. Data Cleaning

Data cleaning means fixing bad data in your data set.

Bad data could be:

- Empty cells
- Data in wrong format
- Wrong data
- Duplicates

a. Cleaning Empty Cells

Empty cells can potentially give you a wrong result when you analyze data.

- `isna()` function: returns **True** to indicate a missing value.
- `notna()` function: returns **False** for missing values.

	state	account_length	churn
0	CA	314	yes
1	CO	?	no
2	?	44	yes

b. Remove Rows

- One way to deal with empty cells is to remove rows that contain empty cells.
- This is usually OK, since data sets can be very big, and removing a few rows will not have a big impact on the result.
- `dropna()` function:
 - ``axis``: 0 (default value, indicates row), 1

- `how`: “any” (default value), “all”
- `thresh`: a row or column that at least X non-missing values will be kept, otherwise will be dropped.

	state	account_length	churn
0	CA	314	?
1	CO	?	?
2	?	44	?

```
[ ]: # Removes rows with any missing values
df.dropna(inplace=True)
```

Note: By default, the `dropna()` method returns a *new* DataFrame, and will not change the original.

If you want to change the original DataFrame, use the `inplace = True` argument:

c. Replace Empty Values

Another way of dealing with empty cells is to insert a *new* value instead. This way you do not have to delete entire rows just because of some empty cells.

	state	account_length	churn
0	CA	314	Yes
1	CO	100	Yes
2	CO	44	no

The `fillna()` method allows us to replace empty cells with a value:

Example

Replace NULL values in the "Calories" columns with the number 130:

```
In [ ]: # Replacing empty values  
df = pd.read_csv('data.csv')  
df["Calories"].fillna(130, inplace = True)
```

d. Replace Using Mean, Median, or Mode

A common way to replace empty cells is to calculate the mean, median or mode value of the column.

Pandas uses the `mean()` `median()` and `mode()` methods to calculate the respective values for a specified column:

Example:

Calculate the MEDIAN, and replace any empty values with it:

```
In [ ]: # Calculating Median

df = pd.read_csv('data.csv')
x = df['Calories'].median()
df['Calories'].fillna(x, inplace = True)
```

e. Handling Duplicate Data

- `drop_duplicates()`: Removes duplicate rows from the DataFrame.

```
[ ]: df.drop_duplicates(inplace=True)
```

6. Data Manipulation with Pandas

- `drop()`: Remove rows or columns from a DataFrame

- **replace()**: Replace values within a DataFrame
- **apply()**: Applying a given function along an axis of a DataFrame or Series.
- **Add a new column from current data.**

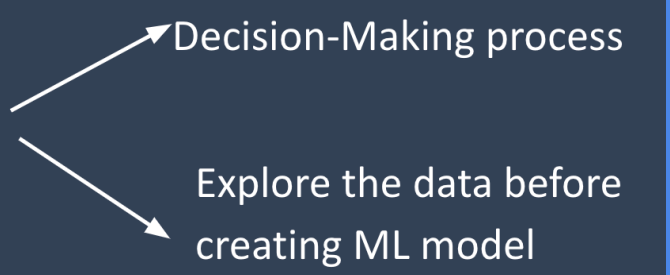
	user_id	account_length	total_orders	total_amt
0	1232as	31	4	156
1	1323wv	0	2	127
2	<u>134pdf</u>	0	3	135
3	342t6ff	23	4	466

7. Data Analysis

Data Analysis: The process of inferring insights, discovering useful information, and drawing results from the data.

- `groupby()`: Groups rows of a DataFrame based on one or more columns.
- `agg()`: Aggregate data within groups created by `groupby()`.
- `reset_index()`: Reset the index of a DataFrame.
- `rename()`: Change the labels (names) of rows or columns in a DataFrame.
- `idxmax()`: Return index of the first occurrence of the maximum over the requested axis.

	total_order	age
0	20	33
1	40	20
2	100	44

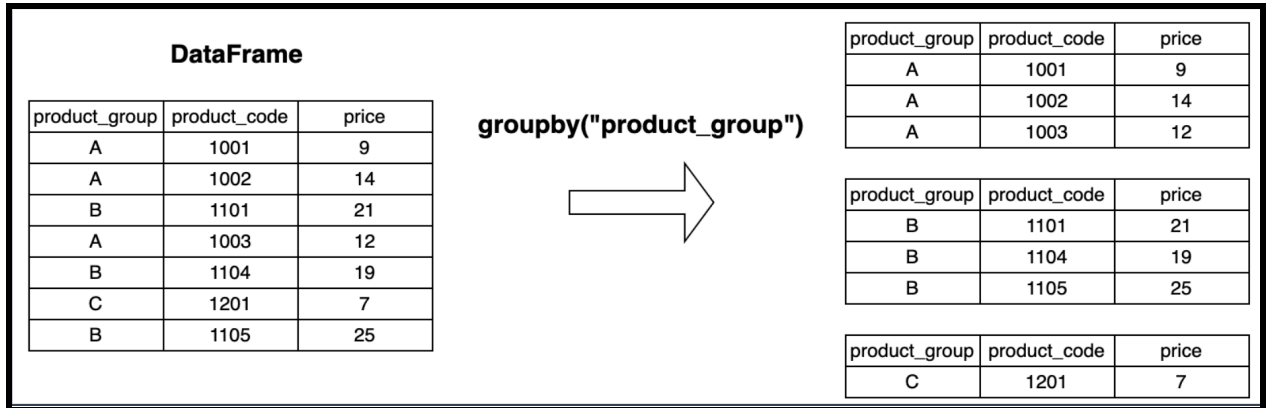


Decision-Making process

Explore the data before creating ML model

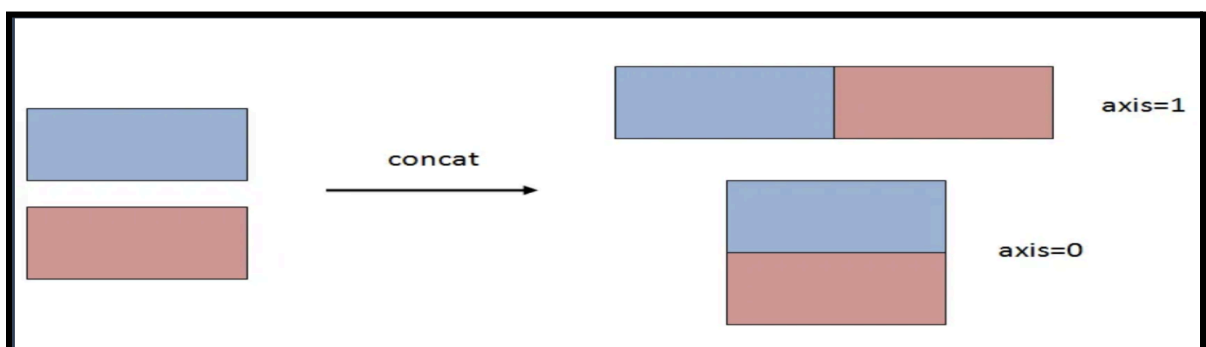
a. Data Analysis with Pandas - `groupby()`, `agg()`

The `groupby` function creates the groups, but they don't provide any information unless we do some aggregations.



b. Combining DataFrames with Pandas - The concat function

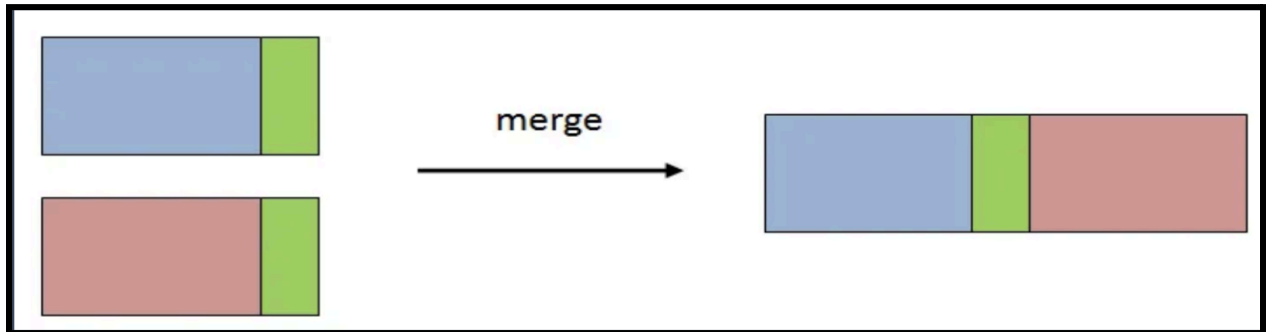
- Concatenation is the process of combining DataFrames along a particular axis (either vertically or horizontally).
- When you concatenate DataFrames, you're essentially stacking them on top of each other (vertically) or side by side (horizontally).
- The resulting DataFrame maintains the original column names and indexes of the individual DataFrames.



c. Combining DataFrames with Pandas - The merge function

- Merging is the process of combining DataFrames based on common columns or indexes.

- It is often used to combine DataFrames with related information, similar to how SQL JOIN operation works.
- Merging allows us to specify which columns to use as keys for merging and how to handle overlapping column names.



d. Basic Statistics

- **mean()**: Calculate the average of a column.

```
[ ]: # Calculate mean of the 'Calories' column
      df['Calories'].mean()
```

- **max()** / **min()**: Get the maximum or minimum values.

```
[ ]: df['Calories'].max() # Returns the maximum value in the 'Calories' column
      df['Calories'].min() # Returns the minimum value in the 'Calories' column
```

e. Filtering by Condition

- Use conditions to filter data.

```
[ ]: # Returns rows where 'Calories' is greater than 100
df[df['Calories'] > 100]
```

f. Grouping Data

- **groupby()**: Group data by one or more columns and perform an aggregation.

```
[ ]: # Group by 'Category' and calculate mean for each group
grouped = df.groupby('Category').mean()
```

8. Modifying DataFrames

- a. **Adding New Columns** : Create a new column based on existing data.

```
[ ]: df['Total'] = df['Price'] * df['Quantity']
```

- b. **Renaming Columns**: Rename the columns of the DataFrame.

```
[ ]: df.rename(columns={'OldName': 'NewName'}, inplace=True)
```

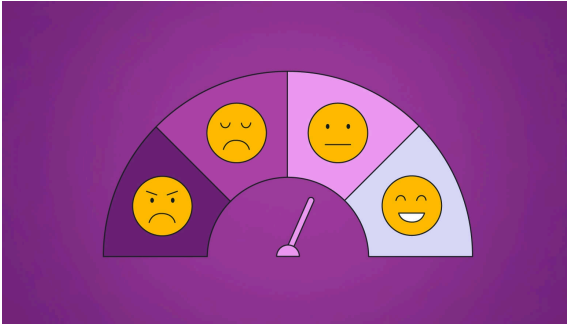
- c. **Sorting Data**: **sort_values()**: Sort the DataFrame by one or more columns.

```
[ ]: df.sort_values(by='Calories', ascending=False, inplace=True)
```

How Do Pandas Help me?

- Pandas will help you handle and analyze large datasets.

- Pandas provide data structures and operations for manipulating numerical tables and time series.



```
import pandas as pd

emotions.set_format(type="pandas")
df = emotions["train"][:]
df.head()
```

	text	label
0	i didnt feel humiliated	0
1	i can go from feeling so hopeless to so damned...	0
2	im grabbing a minute to post i feel greedy wrong	3
3	i am ever feeling nostalgic about the fireplac...	2
4	i am feeling grouchy	3

Pandas Best Practices

- Always inspect your data using `info()` and `head()` before manipulating it.
- Handle missing or duplicate data early in your analysis.
- Use **vectorized operations** instead of explicit loops for faster computation.
- Leverage Pandas' powerful grouping and filtering capabilities for complex analysis.

Matplotlib



Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python.

Pyplot

- **Pyplot** is a module within Matplotlib that provides a MATLAB-like interface. It is the most commonly used submodule for creating plots.
- Typically imported with the alias `plt`.

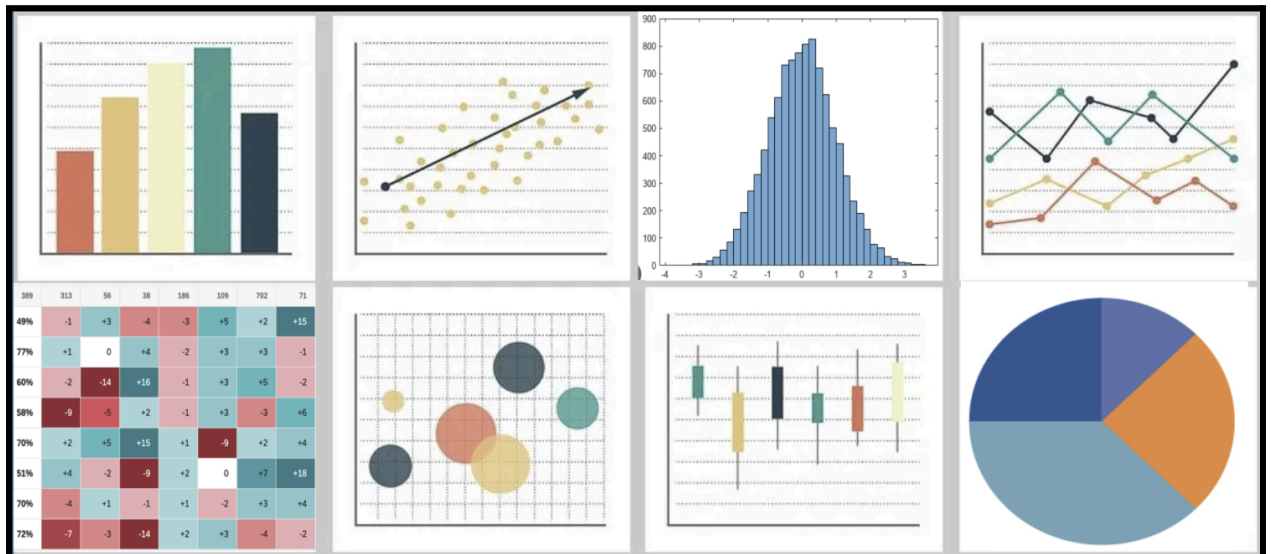
Getting Started with Matplotlib

1. Import Matplotlib

```
In [21]: # Importing Matplotlib  
import matplotlib.pyplot as plt
```

2. Types of Plots

- Line Plot
- Scatter Plot
- Box Plot
- Bar Chart
- Histogram
- Pie Chart
- Heatmap
- Bubble Chart



1. Line Chart

- **Use Case:** Tracking stock prices over time.
- **X-axis:** Time (any ordered series).
- **Y-axis:** Stock Price (A continuous variable showing progression or change over the X-axis).
- **Best Scenario:** To display trends or changes over time.

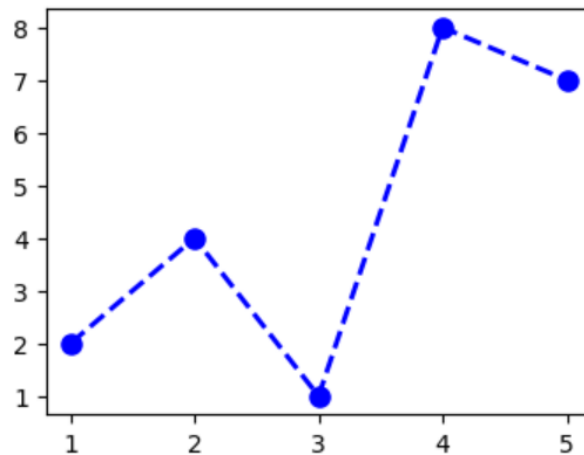


Example:

```
In [22]: x = [1, 2, 3, 4, 5]
         y = [2, 4, 1, 8, 7]

         plt.plot(x, y, marker='o', linestyle='--', color='b', linewidth=2, markersize=8)

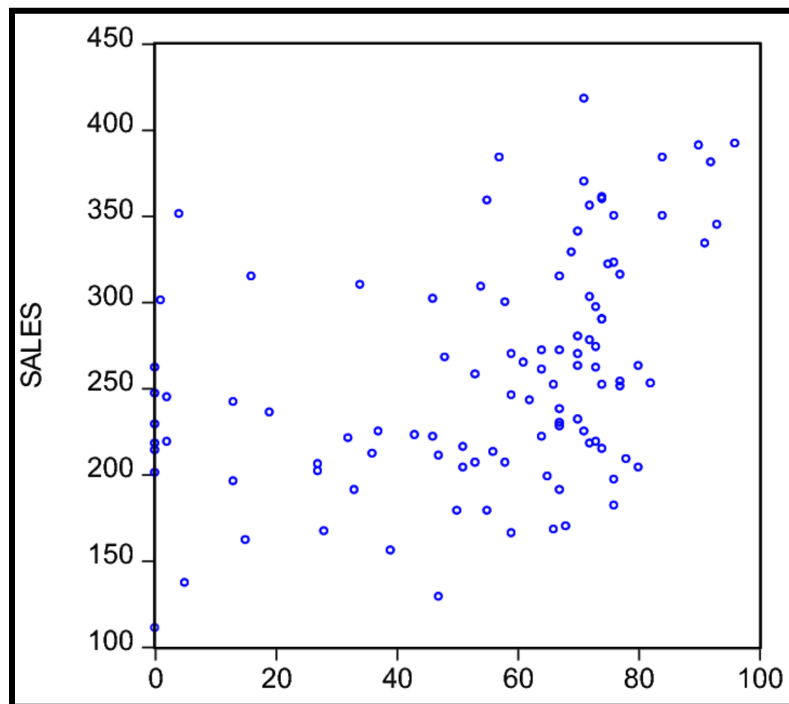
Out[22]: [<matplotlib.lines.Line2D at 0x2b414abf520>]
```



- `marker='o'`: Type of marker (e.g., 'o', 's', '^', etc.)
- `linestyle='--'`: Line style (e.g., '-', '--', ':', etc.)
- `color='b'`: Line color (e.g., 'r' for red, 'g' for green)
- `linewidth=2`: Width of the line
- `markersize=8`: Size of the markers

2. Scatter Chart

- **Use Case:** UberEats analyzing the relationship between advertising spending on Google ads and the revenue generated.
- **X-axis:** Advertising Cost
- **Y-axis:** Sales
- **Best Scenario:** To explore potential relationships or correlations between two continuous variables.

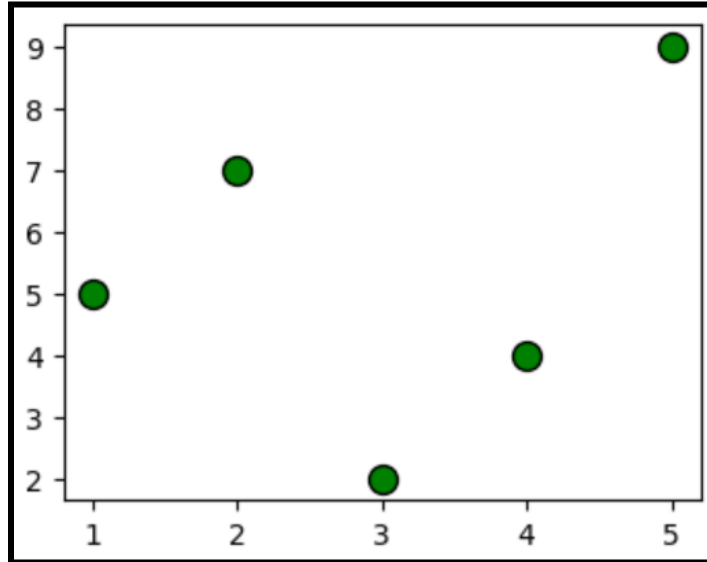


Example:

```
In [23]: x = [1, 2, 3, 4, 5]
         y = [5, 7, 2, 4, 9]

         plt.scatter(x, y, color='g', s=100, edgecolor='black')
```

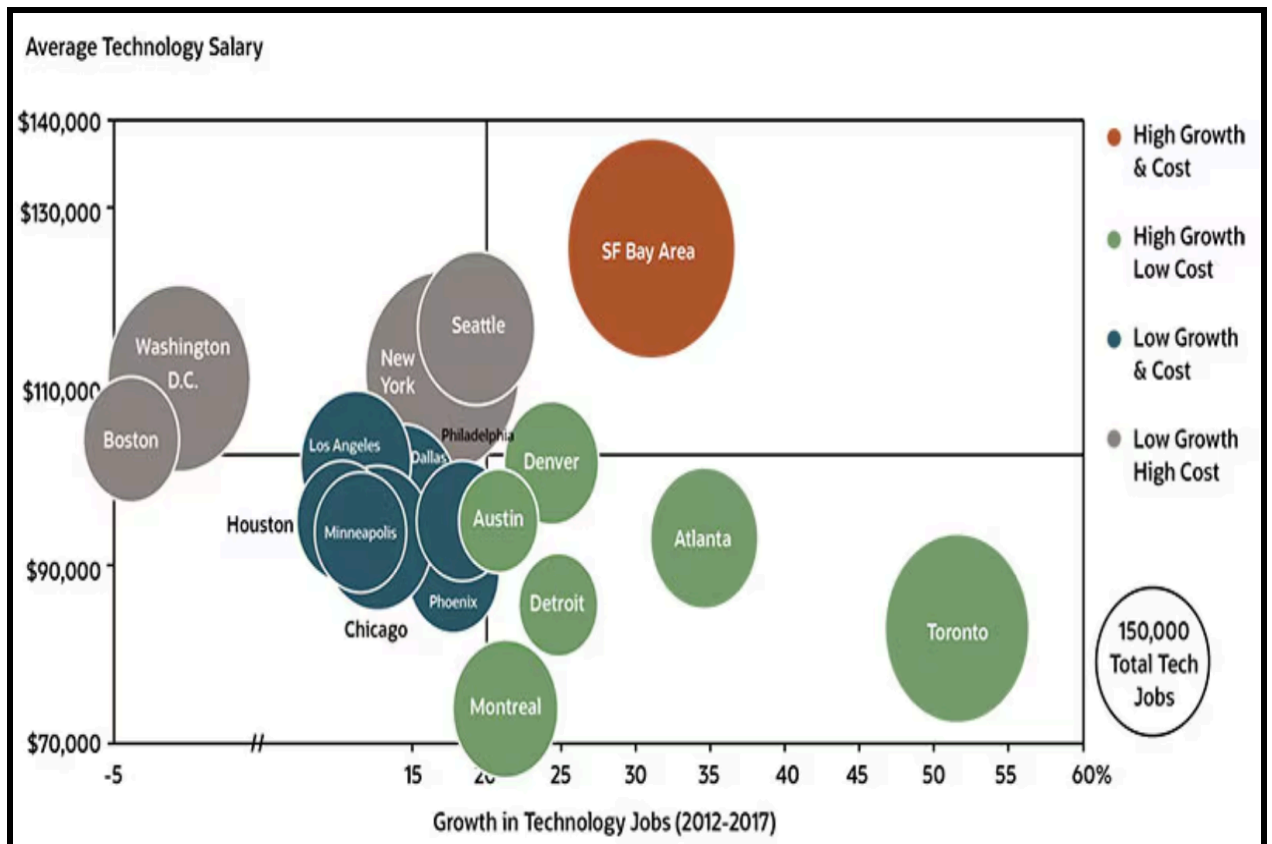
Out[23]: <matplotlib.collections.PathCollection at 0x2b415397460>



- color='g': Color of the points
- s=100: Size of the points
- edgecolor='black': Color of the point edges

3. Bubble Chart

- **Use Case:** To find an affordable city for a tech job.
- **X-axis:** Growth in Tech jobs
- **Y-axis:** Avg Tech Salary
- **Bubble size:** # of job openings
- **Best Scenario:** To find “attractive” clusters considering multiple factors.

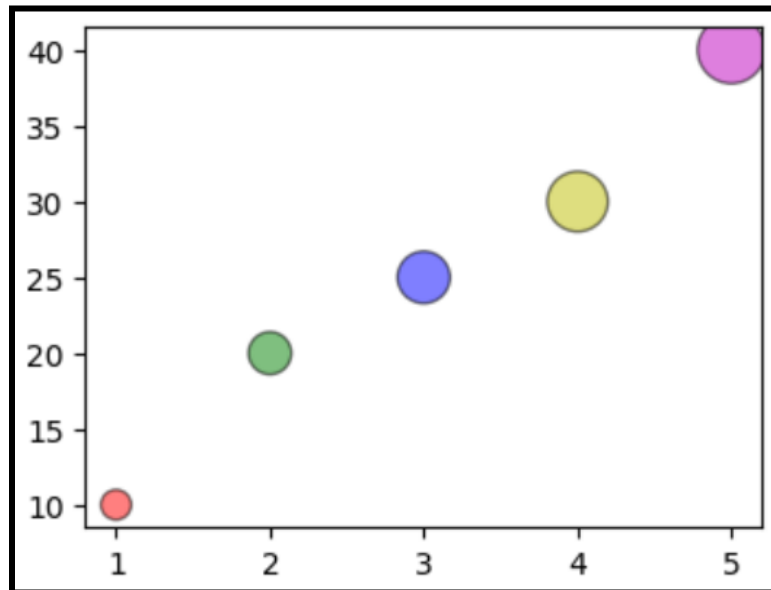


Example:

```
In [24]: x = [1, 2, 3, 4, 5]
y = [10, 20, 25, 30, 40]

area = [100, 200, 300, 400, 500] # Bubble sizes
color = ['r', 'g', 'b', 'y', 'm'] # Bubble colors
plt.scatter(x, y, s=area, c=color, alpha=0.5, edgecolor='black')
```

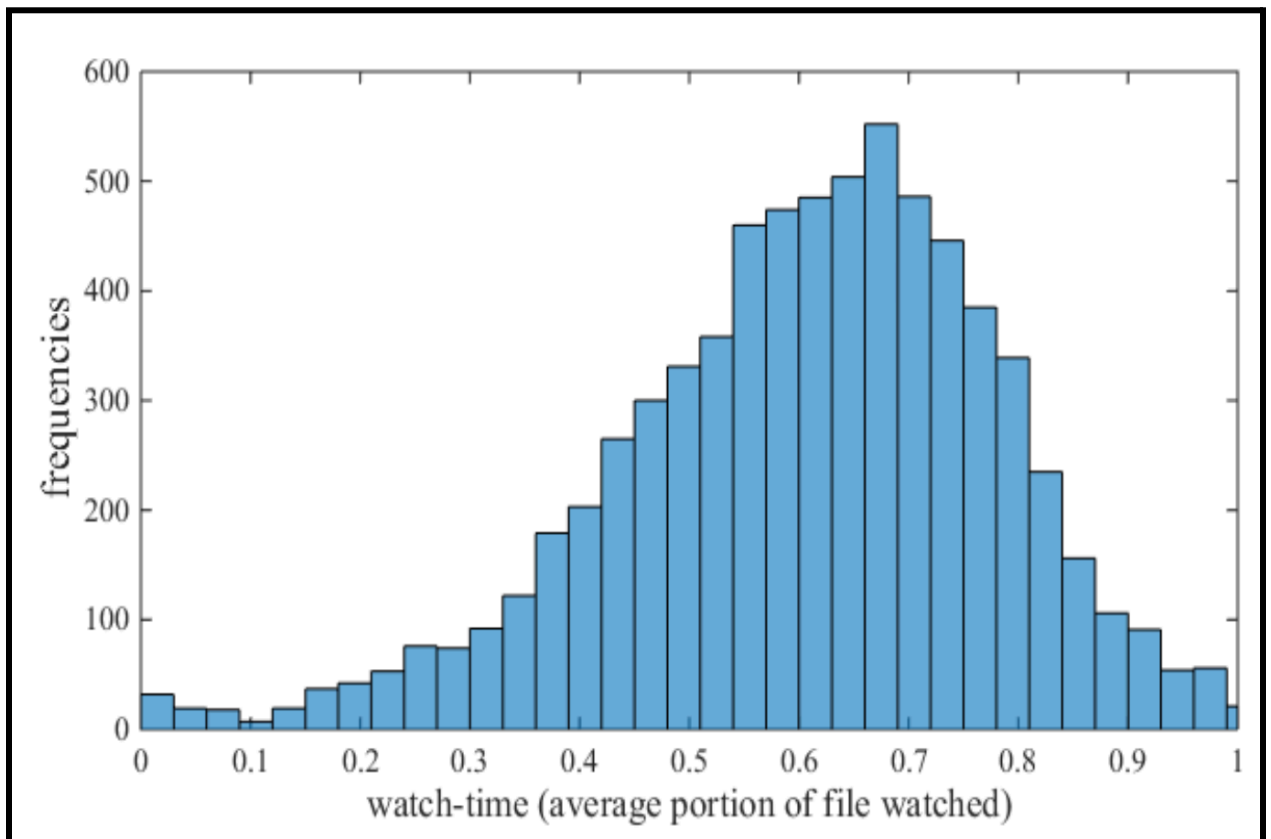
Out[24]: <matplotlib.collections.PathCollection at 0x2b4154a4490>



- `s=area`: Size of the bubbles (array or value)
- `c=color`: Color of the bubbles (array or value)
- `alpha=0.5`: Transparency of bubbles (0 is fully transparent, 1 is opaque)
- `edgecolor='black'`: Color of the bubble edges

4. Histogram

- **Use Case:** Understand the distribution of users watch time on Youtube
- **X-axis:** Watch time on Youtube(continuous variable)
- **Y-axis:** Frequency(# of users)
- **Best Scenario:** To understand the distribution of a dataset.

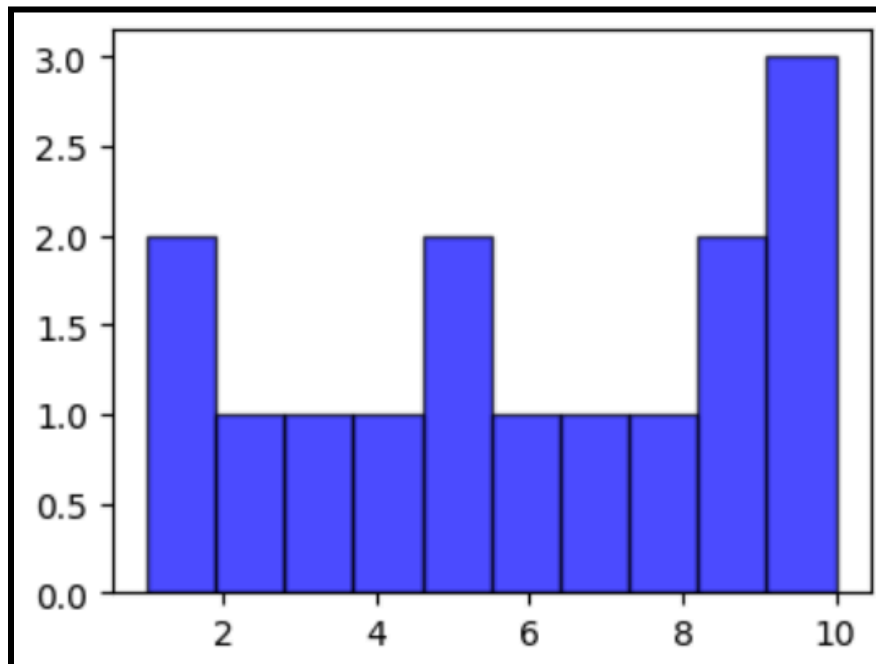


Example:

```
In [25]: data = [1, 1, 2, 3, 4, 5, 5, 6, 7, 8, 9, 9, 10, 10, 10]
```

```
plt.hist(data, bins=10, color='blue', edgecolor='black', alpha=0.7)
```

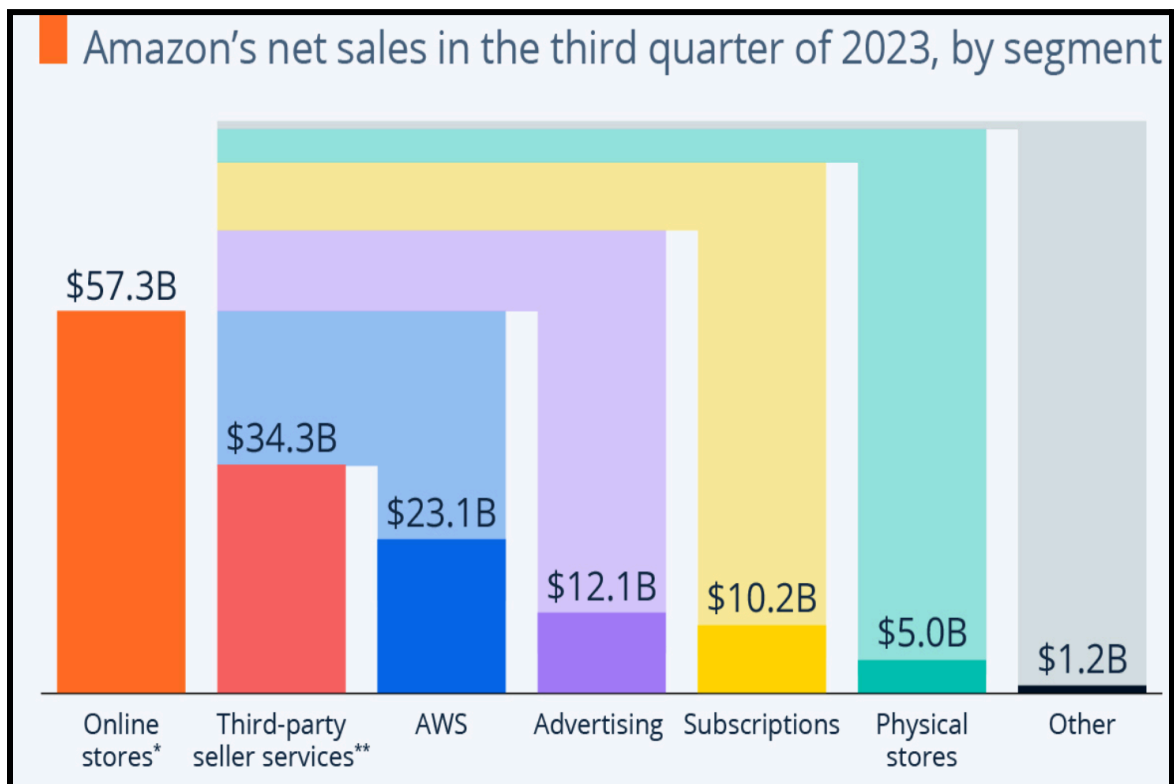
```
Out[25]: (array([2., 1., 1., 1., 2., 1., 1., 1., 2., 3.]),  
array([ 1. ,  1.9,  2.8,  3.7,  4.6,  5.5,  6.4,  7.3,  8.2,  9.1, 10. ]),  
<BarContainer object of 10 artists>)
```



- bins=10: Number of bins
- color='blue': Color of the bars
- edgecolor='black': Color of the bin edges
- alpha=0.7: Transparency of the bars

5. Bar Chart

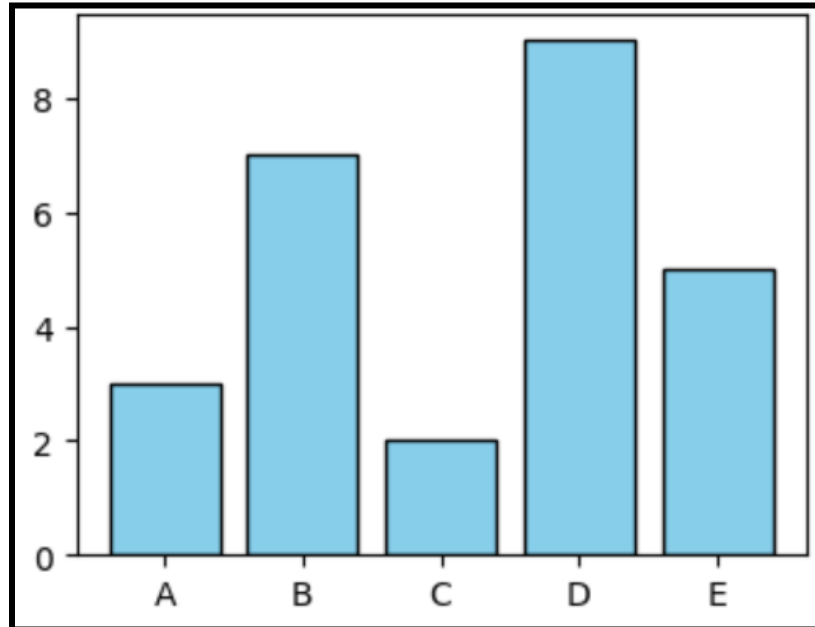
- Use Case: Understand Amazon net sales by segment
- X-axis: segments
- Y-axis: Net sales amount(\$)
- Best Scenario: To compare quantities across categories.



Example:

```
In [26]: x = ['A', 'B', 'C', 'D', 'E']  
         height = [3, 7, 2, 9, 5]  
  
         plt.bar(x, height, color='skyblue', edgecolor='black', width=0.8)
```

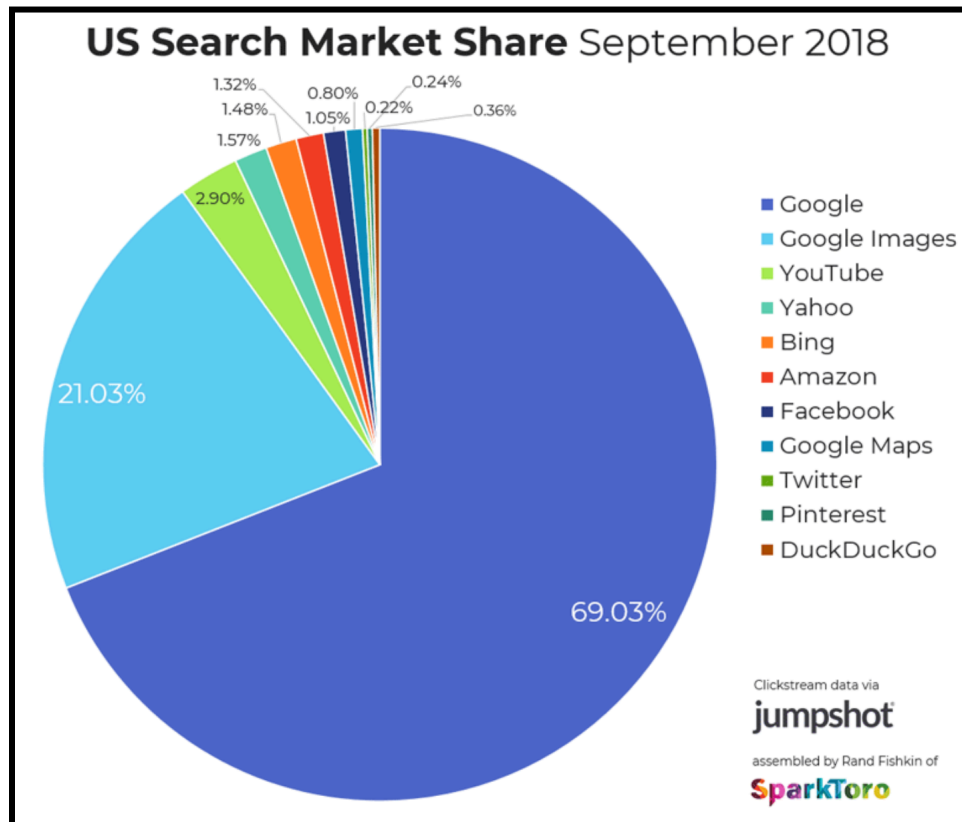
Out[26]: <BarContainer object of 5 artists>



- color='skyblue': Color of the bars
- edgecolor='black': Color of the bar edges
- width=0.8: Width of the bars (default is 0.8)

6. Pie Chart

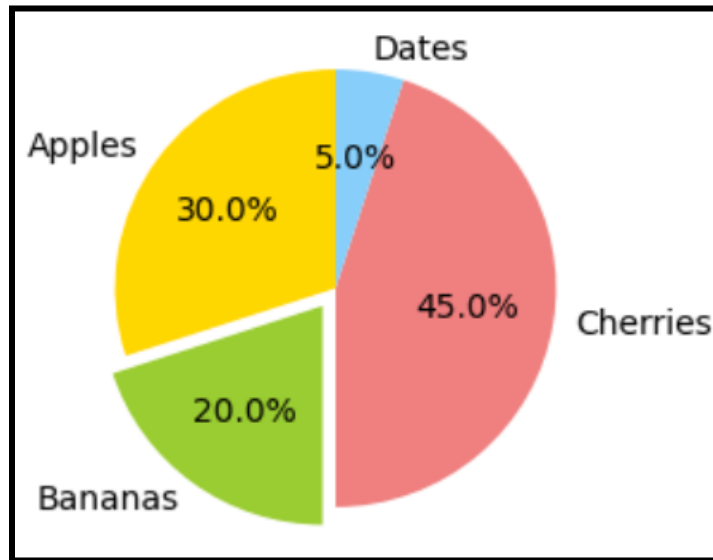
- **Use Case:** Understand the US Search Market Share
- **Best Scenario:** To show parts of a whole in percentage



Example:

```
In [27]: sizes = [30, 20, 45, 5]
labels = ['Apples', 'Bananas', 'Cherries', 'Dates']
colors = ['gold', 'yellowgreen', 'lightcoral', 'lightskyblue']
explode = [0, 0.1, 0, 0] # Explode the 2nd slice (Bananas)

plt.pie(sizes, labels=labels, colors=colors, autopct='%1.1f%%', startangle=90, explode=explode)
```



- labels=labels: Labels for each wedge
- colors=colors: Colors for each wedge
- autopct='%1.1f%%': Display percentage on wedges
- startangle=90: Start angle of the first wedge
- explode=explode: Fraction to "explode" wedges from the center

7. Heatmap

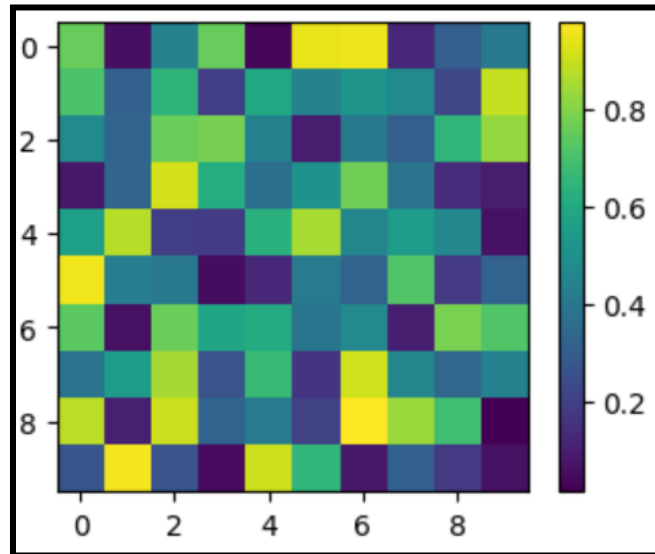
- **Use Case:** Analyzing peak usage times on Instagram throughout the week.
- **Two dimensions:** hours of the day vs. days of the week.
- **Color bar:** Indicating user activity levels.
- **Best Scenario:** Understand how performance changes over the hours of the day and the days of the week.

	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
0	6.00	7.35	11.79	9.82	13.93	7.60	9.10
1	3.73	7.29	7.88	6.62	8.73	5.58	5.63
2	3.44	7.31	5.29	4.23	3.98	3.70	3.06
3	3.67	7.02	3.69	3.12	1.27	2.77	1.94
4	3.81	5.50	1.92	2.65	1.42	3.55	2.29
5	4.54	3.67	1.29	2.48	2.63	5.40	3.58
6	5.52	3.46	3.46	3.67	3.68	6.53	4.33
7	5.75	6.15	7.33	6.23	5.70	6.58	4.85
8	7.94	10.46	10.79	10.40	8.92	6.10	6.25
9	13.58	14.08	14.29	14.13	11.97	7.10	7.38
10	18.77	16.27	18.83	15.70	15.02	10.10	8.23
11	21.00	17.52	24.77	15.25	18.60	12.68	10.10
12	21.13	18.38	31.77	15.27	22.12	13.88	12.60
13	22.54	20.38	36.85	18.55	23.77	14.35	12.81
14	27.02	24.02	37.40	24.62	23.70	14.28	10.15
15	30.52	26.58	33.00	30.13	23.13	13.20	8.15
16	29.42	25.77	25.56	29.78	20.40	11.62	9.46
17	23.67	22.81	19.04	23.28	15.42	11.65	12.63
18	17.88	19.79	15.23	17.08	11.15	13.13	13.92
19	16.38	16.69	14.06	16.17	9.93	13.78	12.04
20	16.04	13.21	14.75	18.90	10.08	13.25	9.58
21	13.77	11.92	14.96	20.88	9.62	12.92	9.69
22	10.48	13.98	14.38	20.17	8.92	12.66	10.98
23	8.00	14.69	13.27	17.85	8.48	11.43	9.58

Example:

```
In [28]: import numpy as np  
data = np.random.rand(10, 10) # 10x10 random matrix  
  
plt.imshow(data, cmap='viridis', interpolation='nearest'); plt.colorbar()
```

Out[28]: <matplotlib.colorbar.Colorbar at 0x2b4155a2530>



- `cmap='viridis'`: Color map for the heatmap (e.g., 'plasma', 'inferno', 'coolwarm')
- `interpolation='nearest'`: Interpolation method (e.g., 'bilinear', 'bicubic')

Matplotlib Subplot

Display Multiple Plots

- With the `subplot()` function you can draw multiple plots in one figure:

The subplot() Function

- The `subplot()` function takes three arguments that describe the layout of the figure.
- The layout is organized in rows and columns, which are represented by the *first* and *second* argument.
- The third argument represents the index of the current plot.

```
In [ ]: plt.subplot(1, 2, 1)  
# The figure has 1 row, 2 columns, and this plot is the first plot
```

```
In [ ]: plt.subplot(1, 2, 2)  
# The figure has 1 row, 2 columns, and this plot is the second plot
```

```
In [29]: import matplotlib.pyplot as plt
import numpy as np

# plot 1
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])

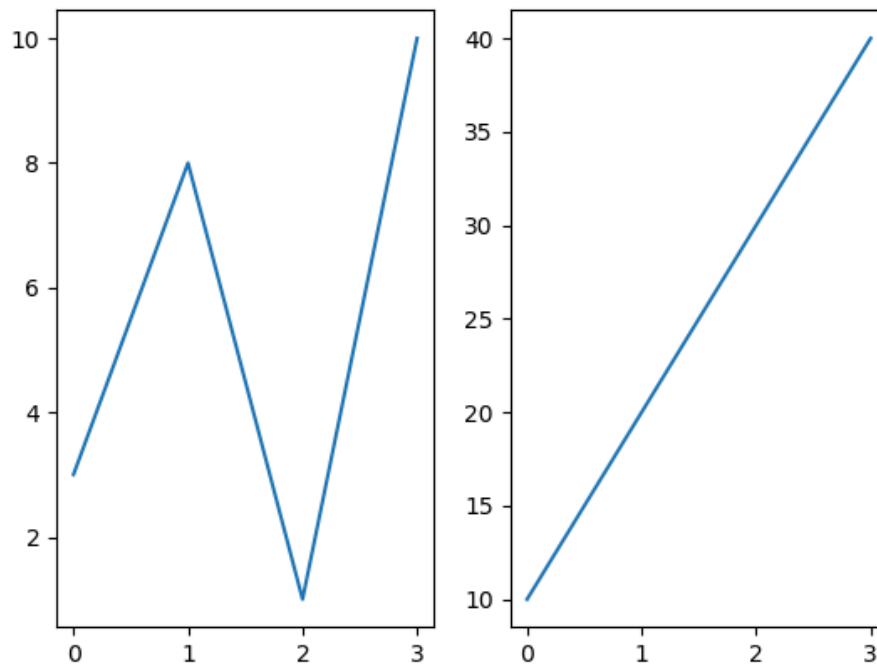
plt.subplot(1, 2, 1)
plt.plot(x, y)

# plot 2
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])

plt.subplot(1, 2, 2)
plt.plot(x,y)

plt.show()
```

Output:



Matplotlib Best Practices

When working with Matplotlib, following some best practices can help improve code readability, efficiency, and the overall quality of your visualizations. Here are key best practices for using Matplotlib effectively:

1. Always Label Your Axes and Titles

Clear labeling of your plots ensures that anyone viewing the visualization can easily interpret the data.

Best Practice:

```
[ ]: plt.plot(x, y)
      plt.xlabel('Time (months)')
      plt.ylabel('Stock Price (USD)')
      plt.title('Stock Price Over Time')
      plt.show()
```

- **Why it helps:** It enhances clarity, making your plot more understandable, especially for sharing with others.

2. Use Legends for Multiple Plots

When visualizing multiple lines, scatter points, or bars, adding a legend helps the audience quickly identify which data corresponds to which plot.

Best Practice:

```
[ ]: plt.plot(x, y1, label='Company A')
      plt.plot(x, y2, label='Company B')
      plt.legend()
      plt.show()
```

- **Why it helps:** Makes the comparison between datasets easier and provides better context for the audience.

3. Set Plot Limits Manually if Necessary

Sometimes, the automatic limits chosen by Matplotlib may not reflect the most meaningful view of your data. You can set the limits manually to focus on the most important parts of your data.

Best Practice:

```
[ ]: plt.xlim([0, 10])
      plt.ylim([0, 100])
```

- **Why it helps:** Makes the comparison between datasets easier and provides better context for the audience.

4. Rotate Tick Labels for Better Readability

When working with time series or categories with long names, rotating tick labels can prevent them from overlapping and make the plot more readable.

Best Practice:

```
[ ]: plt.xticks(rotation=45)
```

- **Why it helps:** Avoids clutter on the axes and ensures the plot is easy to read, especially when working with date labels or long strings.

5. Use Subplots for Multiple Visualizations in One Figure

When you need to show multiple plots side by side or in a grid, using subplots allows you to fit several visualizations into a single figure for comparison.

Best Practice:

```
[ ]: plt.subplot(1, 2, 1)
      plt.plot(x, y1)
      plt.title('Plot 1')

      plt.subplot(1, 2, 2)
      plt.plot(x, y2)
      plt.title('Plot 2')

      plt.tight_layout() # Avoids overlap of subplots
      plt.show()
```

- **Why it helps:** Subplots allow you to compare different datasets or visualizations side by side in a neat and organized manner.

6. Save Figures with High Resolution

When saving plots, especially for reports or publications, it is important to save them in a high resolution.

Best Practice:

```
[ ]: plt.savefig('plot.png', dpi=300)
```

- **Why it helps:** High-quality images look professional and maintain clarity when included in documents or presentations.

7. Customize Colors and Styles

Using different colors and line styles can make your plots more visually appealing and highlight specific data points or trends.

Best Practice:

```
[ ]: plt.plot(x, y1, color='green', linestyle='--')  
     plt.plot(x, y2, color='blue', linestyle='-')
```


Why it helps: It draws attention to specific lines or trends and makes the plot more engaging and easy to distinguish.

8. Use Grids for Better Data Interpretation

Adding a grid to your plots can help guide the reader's eye and make comparisons easier.

Best Practice:

```
[ ]: plt.grid(True)
```

Why it helps: Makes the comparison between datasets easier and provides better context for the audience.

9. Avoid Over-Plotting

Plotting too much data on a single graph can clutter the visualization and make it difficult to extract useful information. Use subplots or different visualizations to break down the data.

Best Practice:

- Use a maximum of 3-4 lines or datasets per plot.
- Use transparency (alpha) to handle overlapping data points.

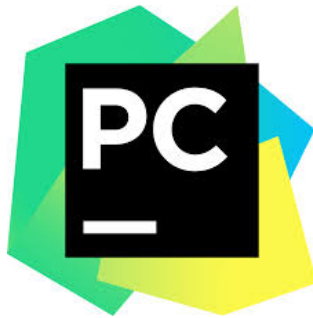
Why it helps: Simplifies the visualization, preventing the audience from getting overwhelmed and ensuring key insights are clear.

How Does Matplotlib Help?

- 1. Visual Communication:** Matplotlib helps transform raw data into visual insights, making it easier for others to understand trends, patterns, and outliers.
- 2. Customization:** You can fully customize plots to meet the requirements of reports, presentations, or dashboards. Everything from line styles, colors, markers, and fonts can be adjusted to fit your needs.
- 3. Integration with Other Libraries:** Matplotlib works seamlessly with other Python libraries like Pandas and NumPy, allowing you to visualize data directly from data manipulation operations.
- 4. Wide Range of Visualizations:** Whether you need simple line charts, scatter plots, or advanced visualizations like heatmaps, Matplotlib provides tools to create all kinds of visual representations.
- 5. Publication Quality Plots:** Matplotlib allows users to generate high-resolution, publication-quality plots suitable for research papers or professional reports.
- 6. Interactive and Static Visualizations:** You can use Matplotlib to create both interactive charts for exploratory analysis and static charts for formal reports.

Python Tools and IDEs

- **PyCharm:** A powerful, full-featured IDE for Python development with intelligent code assistance and tools for debugging and testing.



- **Jupyter Notebook:** An interactive environment that allows you to write, test, and visualize Python code, making it ideal for data analysis and exploration.



- **Google Colab:** A free, cloud-based Jupyter notebook environment that provides access to GPU/TPU for running Python code, perfect for machine learning and data science projects.



- **Visual Studio Code (VS Code):** A lightweight, highly customizable code editor with Python-specific extensions for syntax highlighting, linting, and debugging.



- **IPython:** An enhanced interactive Python shell with additional features like tab completion, syntax highlighting, and magic commands.

Additional Resources

NumPy Resources

1. Official Documentation:

- **NumPy Docs:** <https://numpy.org/doc/stable/>
 - The official and comprehensive documentation of the NumPy library.

2. Books:

- **Python for Data Analysis** by Wes McKinney:
 - This book covers data analysis with Python, including NumPy and Pandas. A highly recommended resource by the creator of Pandas.
- **Numerical Python: A Practical Techniques Approach for Industry** by Robert Johansson:
 - A focused guide on using NumPy for scientific computing.

3. Tutorials and Courses:

- **NumPy Tutorials (W3Schools):**
https://www.w3schools.com/python/numpy_intro.asp
 - A beginner-friendly guide to get started with NumPy.
- **Kaggle NumPy Course:**
<https://www.kaggle.com/learn/numpy>
 - A free, hands-on introduction to NumPy on Kaggle.

4. Community:

- **NumPy GitHub Repository:**
<https://github.com/numpy/numpy>
 - Explore the codebase and contribute to the NumPy library.

Pandas Resources

1. Official Documentation:

- **Pandas Docs:**

- <https://pandas.pydata.org/pandas-docs/stable/>

- The official documentation provides detailed descriptions of Pandas functions, with examples.

2. Books:

- **Python for Data Analysis** by Wes McKinney:

- This book, authored by the creator of Pandas, is an essential guide to using Pandas effectively for data manipulation.

- **Pandas Cookbook** by Theodore Petrou:

- Offers practical recipes for mastering data manipulation with Pandas.

3. Tutorials and Courses:

- **Pandas Documentation Tutorials:**

- https://pandas.pydata.org/pandas-docs/stable/getting_started/tutorials.html

- A series of tutorials hosted by the Pandas project.

- **Kaggle Pandas Course:**

- <https://www.kaggle.com/learn/pandas>

- Kaggle's interactive, beginner-friendly Pandas tutorial.

4. CheatSheets:

- **Pandas Cheat Sheet:**

- https://pandas.pydata.org/Pandas_Cheat_Sheet.pdf

- Official cheat sheet for common Pandas functions.

5. Community:

- **Pandas GitHub Repository:**

- <https://github.com/pandas-dev/pandas>

Matplotlib Resources

1. Official Documentation:

- **Matplotlib Docs:**

- <https://matplotlib.org/stable/contents.html>

- Comprehensive documentation for Matplotlib functions and customization options.

2. Books:

- **Matplotlib for Python Developers** by Aldrin Yim:

- This book teaches you how to use Matplotlib to create a variety of visualizations, with practical examples.

- **Interactive Applications Using Matplotlib** by Benjamin V. Root:

- Ideal for users looking to build interactive visualizations with Matplotlib.

3. Tutorials and Courses:

- **Matplotlib Gallery:**

- <https://matplotlib.org/stable/gallery/index.html>

- A gallery of various types of plots with code snippets for each.

- **Kaggle Matplotlib Course:**

- <https://www.kaggle.com/learn/data-visualization>

- Kaggle's interactive data visualization tutorial using Matplotlib and Seaborn.

4. CheatSheets:

- **Matplotlib Cheat Sheet:**

- <https://github.com/matplotlib/cheatsheets>

5. Community:

- **Matplotlib GitHub Repository:**

- <https://github.com/matplotlib/matplotlib>

Conclusion

Mastering Python, especially libraries like NumPy, Pandas, and Matplotlib, is essential for tasks like machine learning, data analysis, and software development. This cheat sheet offers a detailed yet concise overview of each library. Leverage the best practices and recommended resources to enhance your Python skills and build efficient, robust applications.